

Hungarian Assignment and Sinkhorn Transport for Root Tracking

Benchmark and conclusions report

June 6, 2026

Executive Summary

This report evaluates whether entropic Sinkhorn transport can replace or accelerate Hungarian assignment for tracking polynomial roots across a parameter step. The distinction between the two algorithms is essential. Hungarian returns an exact one-to-one assignment. Sinkhorn returns a soft transport matrix whose row and column sums are correct, but whose entries are continuous rather than binary.

The conclusions are:

- Sinkhorn is attractive when a soft transport cost, confidence matrix, or proposal assignment is acceptable.
- Sinkhorn is not automatically a faster way to compute the exact Hungarian permutation. Making the soft matrix close to a hard assignment can require very small ϵ , log-domain stabilization, and more iterations.
- Parallel Sinkhorn scales well in C because each iteration is dominated by matrix-vector work. This advantage is clearest for large matrices and for difficult or tie-heavy assignment instances.
- For degree-50 root tracking, a compiled Hungarian solver is already fast. Sinkhorn is best used as a confidence-scored proposal for easy continuation steps, with Hungarian as the fallback when confidence checks fail.

Problem Setup

At one parameter value there are old roots z_i^{old} . At the next parameter value there are new roots z_j^{new} . A common tracking cost is the squared distance

$$C_{ij} = |z_i^{\text{old}} - z_j^{\text{new}}|^2.$$

The hard assignment problem asks for a permutation π minimizing

$$\sum_{i=1}^n C_{i,\pi(i)}.$$

The Hungarian algorithm solves this problem exactly and returns a permutation matrix A :

$$\sum_j A_{ij} = 1, \quad \sum_i A_{ij} = 1, \quad A_{ij} \in \{0, 1\}.$$

Sinkhorn instead solves an entropy-regularized transport problem. With uniform marginals, the transport matrix P has row and column sums $1/n$. Scaling by n gives the soft assignment matrix

$$S = nP, \quad \sum_j S_{ij} = 1, \quad \sum_i S_{ij} = 1.$$

The regularization parameter ϵ controls the softness. Large ϵ spreads mass over many possible matches. Small ϵ concentrates mass on low-cost matches and can approach a hard assignment.

Comparison Metrics

The assignment-matrix error is the normalized L^1 distance between the hard Hungarian matrix A and the scaled Sinkhorn matrix S :

$$\text{assign_err}(A, S) = \frac{1}{2n} \sum_{ij} |A_{ij} - S_{ij}|.$$

This metric is 0 when the matrices are identical and is close to 1 when the Sinkhorn matrix places little mass on the Hungarian assignment.

For root tracking, row-wise argmax agreement is often more useful than full matrix error. The Sinkhorn proposal is

$$\hat{\pi}(i) = \arg \max_j S_{ij}.$$

If every row chooses the Hungarian column and the chosen columns are unique, then Sinkhorn proposes the same permutation even when the full soft matrix still has some off-diagonal mass. Confidence is measured by the largest row entry (“top mass”) and by the top-vs-second gap.

Soft Transport Versus Hard Assignment

The following $n = 200$ benchmark uses random one-dimensional point sets and compares a simple Sinkhorn run with a hard Hungarian assignment. It illustrates a central tradeoff: Sinkhorn can compute a soft transport plan quickly while still being far from the hard assignment matrix.

ϵ	Hungarian ms	Sinkhorn ms	speedup	Sinkhorn cost	abs. cost err	assignment overlap	assignment err
0.5	842.157	0.618	1362.7	0.115116	0.113980	0.006740	0.993260
0.1	842.157	0.762	1105.2	0.039495	0.038358	0.011580	0.988420
0.02	842.157	2.417	348.4	0.009973	0.008836	0.022382	0.977618

The timing advantage in this table is attached to a poor hard-assignment match: assignment error remains above 0.97 for all three ϵ values. On this cost scale, reducing assignment error below 0.1 requires log-domain Sinkhorn with epsilon scaling and ϵ around $2 \cdot 10^{-7}$. That sharper computation gives up most of the simple Sinkhorn speed advantage.

Parallel C Sinkhorn

Sinkhorn’s key computational advantage is that each iteration is built from matrix-vector operations. Those row and column scaling passes can be split across threads. Hungarian assignment is much harder to parallelize because it is an augmenting-path combinatorial algorithm with sequential dependencies.

The C benchmark compares a serial shortest-augmenting-path Hungarian solver with a pthread-parallel Sinkhorn solver called from Python through `ctypes`. The benchmark uses three cost families:

- **sorted-1d**: squared distances between sorted random points;
- **dense**: an unstructured random cost matrix;
- **flat**: an all-zero tie-heavy matrix that exposes difficult augmenting-path behavior.

The Sinkhorn runs use fixed 200 iterations at $\epsilon = 0.02$. The C Sinkhorn timing includes building $K = \exp(-C/\epsilon)$, iterating, and computing final errors.

cost kind	n	C Hungarian ms	Sinkhorn 1 thread ms	Sinkhorn 8 thread ms	8 thread / 1 thread	speedup vs Hungarian
sorted-1d	1000	342.561	314.881	54.318	5.80	6.31
sorted-1d	2000	1073.838	1370.680	206.903	6.62	5.19
sorted-1d	3000	1389.212	3168.630	477.274	6.64	2.91
sorted-1d	4000	6651.321	8867.463	1318.831	6.72	5.04
dense	1000	40.245	336.414	61.073	5.51	0.66
dense	2000	204.245	1403.717	222.992	6.29	0.92
dense	3000	552.399	3329.158	490.290	6.79	1.13
dense	4000	1168.892	8865.697	1304.035	6.80	0.90
flat	1000	442.498	339.049	63.596	5.33	6.96
flat	2000	3450.115	1434.381	228.826	6.27	15.08
flat	3000	11335.831	3351.490	490.957	6.83	23.09
flat	4000	27887.982	9553.088	1315.460	7.26	21.20

The data separate three cases. The **sorted-1d** distance matrices are structured, and the dense random matrices do not reliably trigger worst-case Hungarian behavior. The **flat** tie-heavy matrices do: C Hungarian grows from 0.44 seconds at $n = 1000$ to 27.9 seconds at $n = 4000$, while 8-thread Sinkhorn grows from 64 ms to 1.32 seconds. The practical conclusion is conditional. Sinkhorn’s parallelism is valuable for large soft transport problems and for difficult assignment instances, but for exact degree-50 root tracking, compiled Hungarian remains a strong baseline.

Root Tracking Experiment

The synthetic root experiment creates 50 complex roots near the unit circle, then moves them by a small random amount and shuffles the new roots. This mimics the problem of re-indexing roots after a parameter step.

Easy Case: Small Parameter Step

The small-step case uses root motion scale 0.015. Hungarian matches the known synthetic identity perfectly in this case.

Easy degree-50 root tracking. Hungarian time = 0.8 ms; Hungarian vs known truth = 1.

scale	epsilon	sink_ms	argmax_hungarian	argmax_true	unique_cols	assign_error	mean_top	min_top
1.00	0.0098460	0.700	1	1	50	0.319176	0.6808	0.4675
0.50	0.0049230	0.400	1	1	50	0.175442	0.8246	0.5330
0.20	0.0019690	0.433	1	1	50	0.064433	0.9356	0.5860
0.10	0.0009846	0.400	1	1	50	0.029359	0.9706	0.6671
0.05	0.0004923	0.433	1	1	50	0.011273	0.9887	0.8006
0.02	0.0001969	0.400	1	1	50	0.001473	0.9985	0.9643
0.01	0.0000985	0.400	1	1	50	0.000513	0.9995	0.9750

Harder Case: Larger Parameter Step

The harder case uses root motion scale 0.08. Here the nearest-root assignment itself starts to differ from the known synthetic identity. This is the regime where the parameter step may be too large, roots may be close, and assignment confidence matters.

Harder degree-50 root tracking. Hungarian time = 0.7 ms; Hungarian vs known truth = 0.72.

scale	epsilon	sink_ms	argmax_hungarian	argmax_true	unique_cols	assign_error	mean_top	min_top
1.00	0.0060730	0.433	0.98	0.70	49	0.278493	0.7223	0.4167
0.50	0.0030360	0.433	1.00	0.72	50	0.159998	0.8400	0.4960
0.20	0.0012150	0.400	1.00	0.72	50	0.059439	0.9406	0.6475
0.10	0.0006073	0.400	1.00	0.72	50	0.036002	0.9640	0.7295
0.05	0.0003036	0.400	0.98	0.70	49	0.042935	0.9671	0.5000
0.02	0.0001215	0.733	0.98	0.70	49	0.041376	0.9686	0.5000
0.01	0.0000607	0.400	0.96	0.72	48	0.064617	0.9454	0.5000

Root-Tracking Timing

The root-tracking tables report Sinkhorn runtime per ϵ . The timing table in this section puts the Hungarian and Sinkhorn runtimes side by side. The speed ratio is

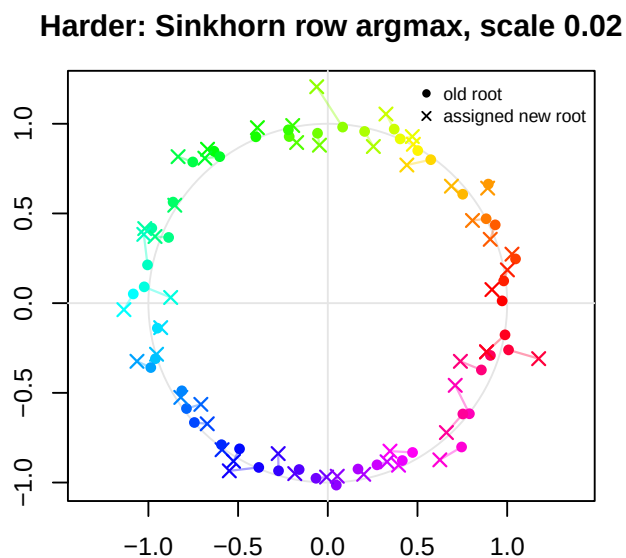
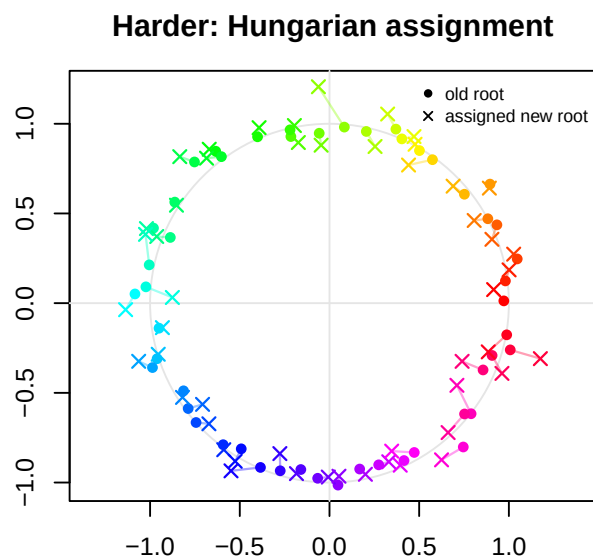
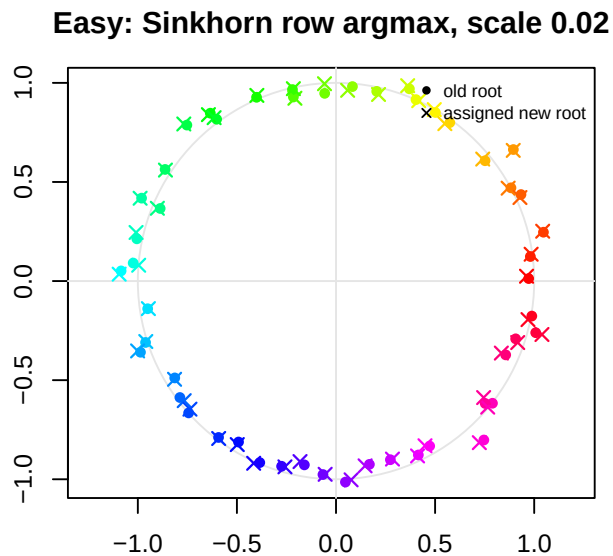
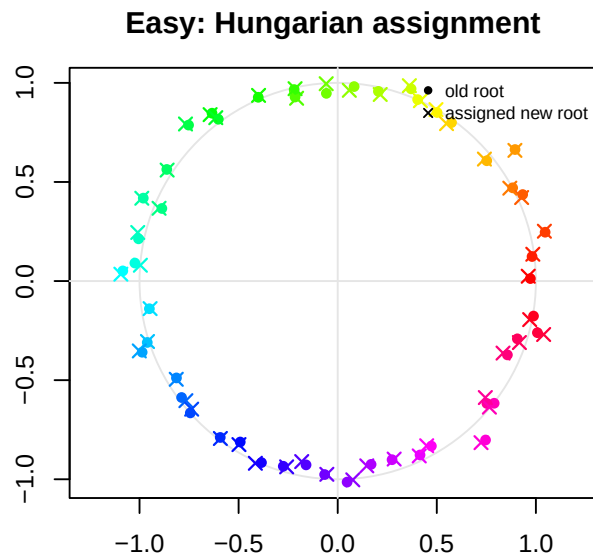
$$\text{speedup} = \frac{\text{Hungarian ms}}{\text{Sinkhorn ms}}.$$

Values greater than 1 mean Sinkhorn was faster for that row.

case	scale	epsilon	hung_ms	sink_ms	speedup	argmax_hungarian	unique_cols
easy	1.00	0.0098460	0.8	0.700	1.14	1.00	50
easy	0.50	0.0049230	0.8	0.400	2.00	1.00	50
easy	0.20	0.0019690	0.8	0.433	1.85	1.00	50
easy	0.10	0.0009846	0.8	0.400	2.00	1.00	50
easy	0.05	0.0004923	0.8	0.433	1.85	1.00	50
easy	0.02	0.0001969	0.8	0.400	2.00	1.00	50
easy	0.01	0.0000985	0.8	0.400	2.00	1.00	50
harder	1.00	0.0060730	0.7	0.433	1.62	0.98	49
harder	0.50	0.0030360	0.7	0.433	1.62	1.00	50
harder	0.20	0.0012150	0.7	0.400	1.75	1.00	50
harder	0.10	0.0006073	0.7	0.400	1.75	1.00	50
harder	0.05	0.0003036	0.7	0.400	1.75	0.98	49
harder	0.02	0.0001215	0.7	0.733	0.95	0.98	49
harder	0.01	0.0000607	0.7	0.400	1.75	0.96	48

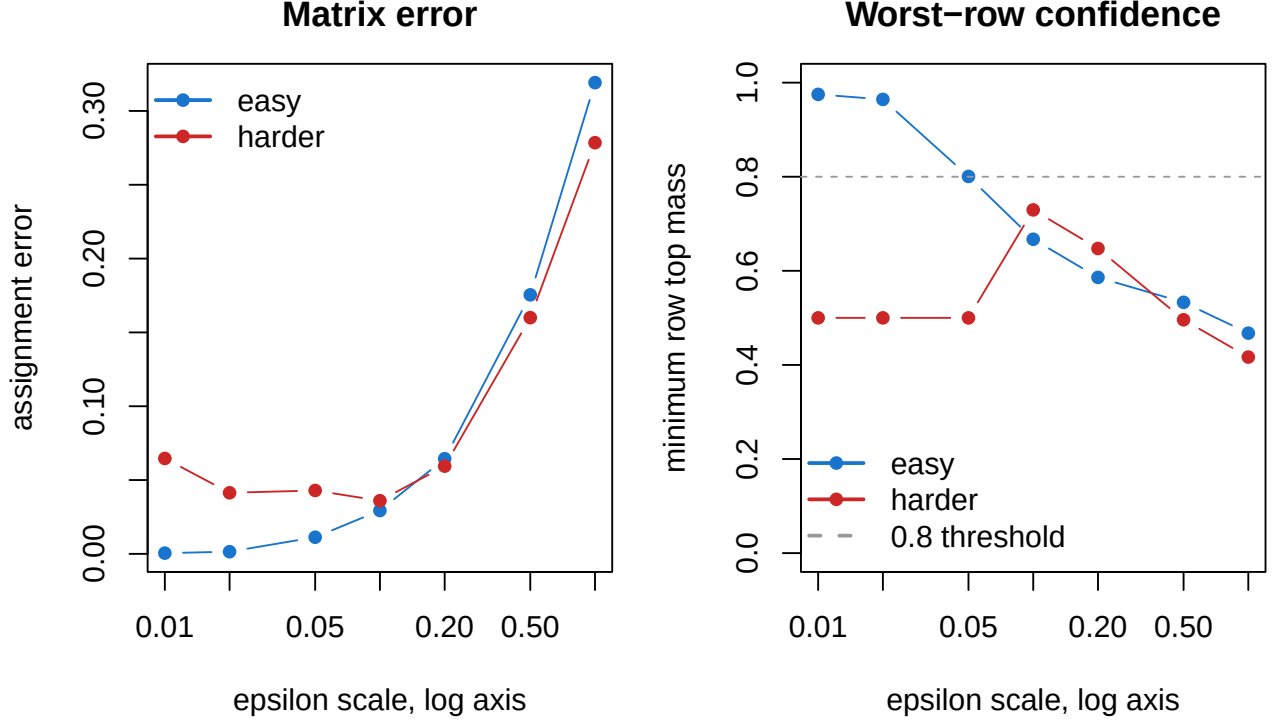
Root-Tracking Plots

Each old root has a fixed rainbow color. The assigned new root is drawn with the same color and connected by a segment. Effective tracking produces short, coherent segments with no obvious crossings or duplicate targets.



Error and Confidence Curves

These curves show that decreasing ϵ sharpens the Sinkhorn matrix. In the easy case, the row argmax agrees with Hungarian across the sweep. In the harder case, duplicate columns and low-confidence rows appear.



Recommendation

For degree-50 root tracking, Sinkhorn can be useful if the roots move in small steps and remain well separated. In the easy synthetic case, 20 primal Sinkhorn iterations were enough for the row argmax to match Hungarian exactly, and the runtime was lower than the R Hungarian timing in the root-tracking table.

The safe workflow is:

1. Build the squared-distance cost matrix between old and new complex roots.
2. Choose ϵ as a fraction of the median nearest-vs-second-nearest cost gap.
3. Run a small fixed number of Sinkhorn iterations.
4. Convert P to $S = nP$, take row argmax, and check uniqueness, top mass, and top-vs-second margin.
5. If checks pass, use the Sinkhorn proposal.
6. If checks fail, fall back to Hungarian or reduce the parameter step.

The confidence checks are:

- all chosen columns are unique;
- every row has high top mass;
- the top-vs-second margin is large.

If exact one-to-one assignment is required with no fallback, Hungarian remains the correct algorithm. Sinkhorn is best viewed as a fast, confidence-scored proposal for easy continuation steps.

A Hungarian Algorithm

The Hungarian algorithm solves the square linear assignment problem

$$\min_{\pi} \sum_{i=1}^n C_{i,\pi(i)},$$

where C_{ij} is the cost of assigning old item i to new item j , and π is a permutation. In root tracking, C_{ij} is usually

$$C_{ij} = |z_i^{\text{old}} - z_j^{\text{new}}|^2.$$

The output is a hard permutation matrix A , where

$$A_{ij} = \begin{cases} 1, & j = \pi(i), \\ 0, & \text{otherwise.} \end{cases}$$

Every row and every column sums to one. This is exactly the structure needed when each old root must be paired with exactly one new root and no new root can be reused.

Dual View

The benchmark uses the standard shortest-augmenting-path form of the Hungarian algorithm. It maintains dual variables u_i and v_j such that reduced costs

$$\tilde{C}_{ij} = C_{ij} - u_i - v_j$$

remain nonnegative on the current equality graph. The algorithm repeatedly adds one row to the matching. For that row, it searches for a cheapest augmenting path through columns using the reduced costs, updates the dual variables, and flips the matching along the path.

Properties

- It returns the exact minimum-cost one-to-one assignment.
- Its usual complexity is $O(n^3)$.
- It gives a hard permutation, not a soft confidence matrix.
- For degree 50, the cost matrix is only 50×50 , so a good compiled Hungarian solver is usually very fast.

Relevance To Root Tracking

Hungarian is the right method when the required output is a guaranteed one-to-one mapping. Its weakness is that it only uses the cost matrix at the current parameter step. If roots are close, collide, or pass near a branch point, the nearest-root assignment can itself become a poor proxy for analytic continuation. In that case, the real fix is often a smaller parameter step or additional continuation information, not a different assignment algorithm.

B Sinkhorn Algorithm

Sinkhorn solves an entropy-regularized version of optimal transport. Given a cost matrix C , uniform marginals $a = b = (1/n, \dots, 1/n)$, and a positive regularization scale ϵ , define the Gibbs kernel

$$K_{ij} = \exp(-C_{ij}/\epsilon).$$

The Sinkhorn algorithm alternates row and column scaling:

$$u \leftarrow \frac{a}{Kv}, \quad v \leftarrow \frac{b}{K^\top u}.$$

After iteration, the transport plan is

$$P = \text{diag}(u)K\text{diag}(v).$$

This plan has approximately the desired row and column sums:

$$P\mathbf{1} \approx a, \quad P^\top \mathbf{1} \approx b.$$

For assignment-style comparison, this report scales P into

$$S = nP.$$

Then S is a soft doubly-stochastic assignment matrix: every row and column sums to approximately one, but entries are continuous rather than binary.

Role Of ϵ

The parameter ϵ controls how soft the plan is:

- Large ϵ : K is diffuse, and P spreads mass across many columns.
- Small ϵ : K concentrates on low-cost entries, and P approaches a hard assignment.

This is why Sinkhorn can be useful for confidence scoring. A row with one very large entry and a large top-vs-second margin is an easy match. A row with several similar entries is ambiguous.

Numerical Stability

For very small ϵ , the entries of $K = \exp(-C/\epsilon)$ can underflow to zero. The stable alternative is log-domain Sinkhorn, which stores the scaling as dual potentials and uses log-sum-exp operations. Epsilon scaling is another standard stabilization strategy: solve an easier large- ϵ problem first, then warm-start smaller ϵ values.

Properties

- Each iteration is mostly matrix-vector multiplication and is easy to vectorize.
- It produces a soft transport plan, not a guaranteed permutation.
- It is useful when a soft cost or confidence matrix is acceptable.
- To recover a hard Hungarian-like assignment matrix, ϵ must be very small; at that point the method can require log-domain stabilization and many iterations.

Relevance To Root Tracking

For root tracking, Sinkhorn can be used as a fast proposal:

$$\hat{\pi}(i) = \arg \max_j S_{ij}.$$

This proposal is usable when all selected columns are unique and every row is confident. If any column is selected twice, if a row has low top mass, or if the top-vs-second margin is small, then the step should fall back to Hungarian or the parameter step should be reduced.